Project Proposal: Cache-Aware Graph Reordering for Efficient Graph Neural Network Execution on GPUs

Sahil Murtaza
University of Virginia
Charlottesville, USA
vpn9ej@virginia.edu

Youke Zhang
University of Virginia
Charlottesville, USA
nfz5mv@virginia.edu

Zach Yu
University of Virginia
Charlottesville, USA
rqx6rs@virginia.edu

Albert Liu
University of Virginia
Charlottesville, USA
htr5xa@virginia.edu

Abstract

Graph Neural Networks (GNNs) suffer from significant hardware inefficiencies during training due to the irregular, sparse structure of real-world graphs. During neighbor aggregation, GPU threads must fetch feature vectors from non-contiguous memory locations, leading to poor spatial locality and high cache miss rates in the L2 cache and HBM subsystem. This proposal outlines a project to investigate, implement, and benchmark *cache-aware node reordering* as a lossless pre-processing step for GNN training. Beyond evaluating existing reordering algorithms (Reverse Cuthill-McKee and METIS), we propose two novel contributions: (1) a *two-level hybrid reordering* that first partitions the graph via METIS and then applies RCM within each partition, composing block-diagonal and band-diagonal structure into a single permutation; and (2) a *cache-size-aware partition selection* rule that analytically derives the optimal number of METIS partitions k^* from the GPU's L2 cache capacity, eliminating the need for exhaustive hyperparameter sweeps. We will evaluate these methods on Open Graph Benchmark datasets, measuring GPU L2 cache hit rates and epoch training latency. Because reordering is a relabeling that commutes with all GNN operators, prediction accuracy is guaranteed to remain identical to the baseline.

Keywords

Graph Neural Networks, GPU Architecture, Cache Optimization, Node Reordering, Sparse Matrix Bandwidth, Graph Partitioning

1 Introduction: The Problem

The dominant bottleneck in accelerating Graph Neural Networks (GNNs) on GPUs is the *memory wall*. Modern GPU architectures are optimized for dense, contiguous general matrix multiplications (GEMMs), which exploit memory coalescing to saturate compute throughput. Real-world graphs, however, are highly sparse and topologically irregular, causing the core message-passing operations of GNNs to devolve into *scatter–gather* memory access patterns that defeat HBM coalescing and L2 cache reuse.

Message-Passing as Sparse Computation. Consider a graph $G = (V, E)$ with $n = |V|$ nodes, adjacency matrix A, and node features $H^{(l)} \in \mathbb{R}^{n \times d_l}$ at layer l. Each GNN layer computes $H^{(l+1)} = \sigma(\tilde{A}\, H^{(l)}\, W^{(l)})$, where $\tilde{A} = D^{-1/2} A D^{-1/2}$ is the normalized adjacency [7] and $W^{(l)}$ is a learnable weight matrix. The $\tilde{A}H^{(l)}$ term is a sparse matrix–dense matrix multiplication (SpMM). In Compressed Sparse Row (CSR) storage, row i of $\tilde{A}$ lists the column indices of node i's neighbors; these indices determine which rows of $H^{(l)}$ must be fetched during aggregation.

Why Random Orderings Cause Cache Misses. With arbitrary node ordering, connected nodes can have distant indices, so CSR column indices in any row scatter across memory. The GPU must fetch rows of $H^{(l)}$ from non-contiguous addresses, triggering L2 capacity misses. For large graphs (e.g. `ogbn-products`: n=2.4M, d=100), $H^{(l)}$ occupies ~960 MB—far exceeding the 40 MB L2 cache on an A100—so most accesses go to HBM, stalling the streaming multiprocessors.

Project Goal. We aim to find a permutation π of the node index set such that connected nodes receive *nearby* indices, clustering the non-zeros of $\tilde{A}$ near the diagonal and improving spatial locality during SpMM.

2 Mathematical Formulation of Graph Reordering

2.1 Permutations and the Reordered System

Given a permutation π on $\{1, \ldots, n\}$ with associated permutation matrix P ($P_{ij} = 1$ iff $\pi(i) = j$), the reordered adjacency matrix is the orthogonal similarity transform

$$A' = P\, A\, P^{\top}. \tag{1}$$

Since GNN message-passing is equivariant to node relabeling, reordering preserves all structural invariants and classification accuracy—it is a *lossless* transformation.

2.2 Bandwidth Minimization

The **bandwidth** of a matrix A under ordering π is

$$B_\pi(A) = \max_{(i,j) \in E} \left| \pi(i) - \pi(j) \right|. \tag{2}$$

A small bandwidth means every non-zero entry $A'_{\pi(i),\pi(j)}$ lies within $B_\pi(A)$ positions of the diagonal. During SpMM, this guarantees that the column indices in any CSR row span an interval of width at most $B_\pi(A)$, so the rows of $H'^{(l)}$ accessed by a single aggregation step lie within a contiguous memory region of size $B_\pi(A) \cdot d_l$ floats, improving cache line reuse.

Bandwidth minimization seeks

$$\pi^* = \arg\min_{\pi} \; B_\pi(A), \tag{3}$$

which is NP-hard in general [8]. In practice, heuristics such as Cuthill-McKee (CM) and its reversal (RCM) [3] provide near-optimal results for many graph families.

2.3 The Reverse Cuthill-McKee (RCM) Algorithm

The Reverse Cuthill-McKee algorithm [3] is a BFS-based heuristic that traverses vertices in ascending-degree order from a peripheral seed node and then reverses the resulting ordering to minimize bandwidth. It runs in $O(|E| \log |E|)$ time and produces a band-diagonal A'.

2.4 The METIS Graph Partitioning Objective

METIS [6] partitions V into k disjoint subsets $V_1, \ldots, V_k$ by minimizing the **edge cut**:

$$\text{EdgeCut}(V_1, \ldots, V_k) = \left|\{(i,j) \in E : i \in V_p,\; j \in V_q,\; p \neq q\}\right|, \tag{4}$$

subject to a balance constraint $|V_p| \leq (1+\epsilon)\, n/k$ for all p. Minimizing the edge cut maximizes the fraction of edges whose endpoints share the same partition. We construct the reordering permutation π_{METIS} by numbering all nodes in V_1 first (indices $1, \ldots, |V_1|$), then all nodes in V_2, and so on:

$$\pi_{\text{METIS}}(v) = \sum_{q=1}^{p-1} |V_q| + \text{rank}(v, V_p), \qquad v \in V_p, \tag{5}$$

where $\text{rank}(v, V_p)$ is the position of v within partition V_p (ordered arbitrarily, or recursively by a nested bisection). This produces a block-diagonal-like structure in A': intra-partition edges form dense diagonal blocks of size $\approx n/k$, while the minimized inter-partition edges appear as sparse off-diagonal entries. During SpMM, the GPU processes rows within a partition whose column indices fall predominantly within the same block, causing the required rows of $H'^{(l)}$ to reside in a contiguous $\approx (n/k) \cdot d_l$-float region that fits well in L2 cache.

The number of partitions k is a tunable hyperparameter: larger k yields smaller blocks (better L2 fit) but more inter-partition edges (more off-diagonal scatter).

3 Proposed Novel Contributions

While RCM and METIS are well-studied individually, prior work applies them in isolation and treats the METIS partition count k as a hyperparameter to be swept blindly. We propose two novel contributions that address these limitations.

3.1 Contribution 1: Two-Level Hybrid Reordering

RCM produces a globally band-diagonal A' but cannot exploit community structure; METIS produces block-diagonal A' but leaves the *internal ordering* within each block arbitrary. We propose composing the two into a single **two-level hybrid permutation**:

$$\pi_{\text{hybrid}} = \pi_{\text{RCM}}^{\text{local}} \circ \pi_{\text{METIS}}. \tag{6}$$

The pipeline operates in two stages:

(1) **Stage 1 (METIS).** Partition V into k subsets $V_1, \ldots, V_k$ via METIS, minimizing the edge cut (Eq. (4)). Construct the inter-partition permutation π_{METIS} that groups nodes by partition (Eq. (5)).

(2) **Stage 2 (Intra-partition RCM).** For each partition V_p, extract the induced subgraph $G[V_p]$ and compute its local RCM ordering π_p^{RCM}. The local permutation reassigns indices within the contiguous block allocated to V_p, so the global index of node $v \in V_p$ becomes:

$$\pi_{\text{hybrid}}(v) = \underbrace{\sum_{q=1}^{p-1} |V_q|}_{\text{block offset}} + \pi_p^{\text{RCM}}\big(\text{rank}(v, V_p)\big). \tag{7}$$

Why this helps. After Stage 1, the reordered matrix A' has a block-diagonal structure: most non-zeros in block p reference columns within the same block. After Stage 2, the non-zeros *within* each block are further concentrated near that block's local diagonal. The combined effect is that, during SpMM on rows belonging to partition V_p, nearly all column accesses fall within a narrow band of the p-th diagonal block. Concretely, the *local bandwidth* of block p satisfies:

$$B_p = \max_{(i,j) \in E(G[V_p])} \left|\pi_p^{\text{RCM}}(i) - \pi_p^{\text{RCM}}(j)\right| \;\leq\; B_{\text{global_RCM}}, \tag{8}$$

where $B_{\text{global_RCM}}$ is the bandwidth achieved by applying RCM to the entire graph. The local bandwidth is typically much smaller because the subgraph $G[V_p]$ has fewer nodes and is denser (due to METIS's edge-cut minimization). This means the memory footprint accessed per partition is both *small* (block size $\approx n/k$) and *narrow* (local bandwidth $B_p \ll n/k$), giving a multiplicative improvement in cache reuse over either method alone.

The composition preserves correctness since both π_{METIS} and π_p^{RCM} are permutations, and their composition is also a permutation. Complexity is $O(|E|) + \sum_{p=1}^{k} O(|E_p| \log |E_p|) = O(|E| \log |E|)$.

3.2 Contribution 2: Cache-Size-Aware Partition Selection

Existing approaches treat the METIS partition count k as a free hyperparameter and resort to grid search. We derive k analytically from the hardware specification.

Derivation. During SpMM, processing the rows belonging to partition V_p requires accessing the feature vectors $h_j^{(l)}$ for all neighbors j of nodes in V_p. In the ideal case (zero edge cut), all such neighbors lie within V_p itself, so the working set is the feature sub-matrix $H_p^{(l)} \in \mathbb{R}^{|V_p| \times d_l}$. For this sub-matrix to fit entirely in the GPU's L2 cache of capacity C bytes, we need:

$$|V_p| \cdot d_l \cdot b \;\leq\; C, \tag{9}$$

where b is the number of bytes per element (e.g. $b = 4$ for `float32`). Under the balanced partition assumption $|V_p| \approx n/k$, substitution yields the **cache-optimal partition count**:

$$\boxed{k^* = \left\lceil \frac{n \cdot d_l \cdot b}{C} \right\rceil.} \tag{10}$$

Practical refinement. Equation (10) provides a lower bound on k under the (optimistic) assumption of zero edge cut. In practice, inter-partition edges cause some out-of-block accesses, so we introduce a headroom factor $\alpha > 1$ to reserve cache capacity for these spill accesses:

$$k^*_{\text{practical}} = \left\lceil \alpha \cdot \frac{n \cdot d_l \cdot b}{C} \right\rceil, \qquad \alpha \in [1.0,\ 1.5]. \tag{11}$$

We will experimentally calibrate α by measuring the L2 hit rate at $k = k^*$ and $k = \lceil 1.25\, k^* \rceil$ and selecting the value that maximizes the hit rate. This replaces the blind sweep over $k \in \{4, 8, 16, 32, 64\}$ with a principled, hardware-aware choice that adapts automatically to any GPU and any graph.

Integration with Hybrid Reordering. The cache-size-aware k^* from Eq. (10) feeds directly into the hybrid pipeline (§3.1) as the partition count for Stage 1. The full novel pipeline is therefore: compute k^* from hardware specs → METIS partition with k^* blocks → intra-block RCM → train.

4 Existing Literature

Classical bandwidth reduction heuristics such as Cuthill-McKee [3] and multilevel graph partitioners like METIS [6] form the algorithmic foundation for our work. In the context of graph analytics, Rabbit Order [1] introduced hierarchical community-based reordering that maps graph topology to cache hierarchy levels. On the deep learning side, GNNAdvisor [10] demonstrated that 2D workload management with lightweight node renumbering improves spatial locality for GNN CUDA kernels, and Bayer et al. [2] provided a systematic empirical study showing that even simple reordering strategies can reduce PyTorch Geometric training time by up to 30%. Our work differs from all of the above by composing partitioning and bandwidth reduction into a single two-level permutation and by deriving the partition count analytically from hardware specifications.

5 Methodology

5.1 Datasets

We will use medium-to-large-scale datasets from the Open Graph Benchmark (OGB) [5], selected to expose hardware memory bottlenecks at two different scales:

- **`ogbn-arxiv`** (169K nodes, 1.1M edges): A citation network of arXiv papers with 128-dimensional word2vec features. Small enough for rapid iteration but large enough to observe cache effects.
- **`ogbn-products`** (2.4M nodes, 61.8M edges): An Amazon co-purchasing network. At this scale, the feature matrix $H \in \mathbb{R}^{2.4\text{M}\times 100}$ occupies ~960 MB, substantially exceeding the 40 MB L2 cache on an A100 GPU.

Data will be acquired programmatically via the `torch_geometric.datasets` API.

5.2 GNN Architectures (Workload Models)

We benchmark reordering across two architectures that exercise different memory access patterns:

(1) **GraphSAGE [4]:** Samples a fixed-size neighborhood per node; the sampled column indices determine the memory footprint per aggregation. Reordering affects the spatial coherence of sampled indices.
(2) **Graph Attention Network (GAT) [9]:** Computes attention coefficients α_{ij} over all neighbors before aggregation, requiring *two* accesses to each neighbor's features (once for attention, once for weighted sum). This doubles the cache pressure per edge, amplifying the benefit of locality.

5.3 Hardware Profiling

To establish a causal link between reordering and performance, we will profile the compiled CUDA kernels using **NVIDIA Nsight Compute (`ncu`)**. Key metrics include:

- `l2_cache_hit_rate`: fraction of L2 sector requests serviced without going to HBM.
- `dram_read_throughput`: sustained HBM read bandwidth, which should *decrease* after reordering if fewer cache misses occur.
- `sm_occupancy`: warp occupancy, which may increase if fewer warps are stalled on memory.

6 Evaluation Plan

The project will be evaluated through a controlled ablation study comparing five configurations: (1) baseline (original ordering), (2) RCM-only (via `scipy.sparse.csgraph`), (3) METIS-only with swept k (via `pymetis`), (4) METIS with cache-aware k^*, and (5) hybrid reordering. For each, we construct $A' = PAP^\top$, $H' = PH$, $y' = Py$ and rebuild PyG edge indices.

- **Primary Systems Metric — Epoch Training Time.** Mean wall-clock time per epoch (ms/epoch), averaged over 50 epochs after a 10-epoch warmup. We target a 15–30% latency reduction for baseline methods [2], with an additional improvement from the hybrid approach.
- **Secondary Hardware Metric — L2 Cache Hit Rate.** Measured via `ncu` on the SpMM kernels. The key novelty test is whether hybrid reordering achieves a *higher* hit rate than either RCM-only or METIS-only.
- **Cache-Aware k^* Validation.** We will compare the L2 hit rate at the analytically derived k^* against the best k found by exhaustive sweep.
- **ML Sanity Check.** Node classification accuracy must remain *identical* to the baseline within floating-point tolerance, confirming the reordering is lossless.

7 Project Timeline

The project is organized into five sequential phases:

(1) **Environment and Baselines.** Set up the GPU profiling environment, acquire OGB datasets, implement baseline GraphSAGE and GAT models, and record initial timing and cache metrics.
(2) **Baseline Reordering.** Implement the RCM-only and METIS-only pipelines, validate correctness by checking that $A' = PAP^\top$ preserves the edge set.

(3) **Novel Methods.** Implement the cache-size-aware partition selection (Eq. (10)) and the two-level hybrid pipeline (Eq. (6)). Calibrate α.
(4) **Execution and Profiling.** Run all five configurations at scale, collect ncu hardware traces, and build the full ablation table.
(5) **Analysis and Reporting.** Visualize adjacency matrix sparsity patterns (before/after), plot the cache hit rate vs. training time Pareto frontier, and assemble the final report.

References

[1] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Imai. 2016. Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 22–31. doi:10.1109/IPDPS.2016.110

[2] Moritz Bayer et al. 2024. Can Graph Reordering Speed Up Graph Neural Network Training? *arXiv preprint arXiv:2409.11236* (2024).

[3] Elizabeth Cuthill and James McKee. 1969. Reducing the Bandwidth of Sparse Symmetric Matrices. In *Proceedings of the 1969 24th National Conference*. ACM, 157–172.

[4] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *Advances in Neural Information Processing Systems (NeurIPS)* (2020).

[6] George Karypis and Vipin Kumar. 1998. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. *SIAM Journal on Scientific Computing* 20, 1 (1998), 359–392.

[7] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *Proceedings of the 5th International Conference on Learning Representations (ICLR)* (2017).

[8] Christos H. Papadimitriou. 1976. The NP-Completeness of the Bandwidth Minimization Problem. *Computing* 16 (1976), 263–270.

[9] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.

[10] Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie, and Yufei Ding. 2021. GNNAdvisor: An Adaptive and Efficient Runtime System for GNN Acceleration on GPUs. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. 515–531.